Serialization and networking

Code generation · equality · HTTP · retry

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Week 8

TODAY

What this lab is for

Generate JSON code

Writes fromJson and toJson for you.

Compare by value

Same fields make two todos equal.

Fetch live data

dio loads the list from a public API.

Retry with backoff

Delay and jitter survive slow servers.

TIME

2 hours

SUBMIT

Push to `admd-labs`, branch `lab-4`, before Lab 5

SETUP

Where your code goes

1. Continue in `admd-labs`. Branch: `git checkout -b lab-4`.
2. Two new files: `lib/models/todo.dart` and `lib/api/todo_api.dart`.
3. Add the packages with the two commands below.
4. After each model change:
   `dart run build_runner build --delete-conflicting-outputs`

```
flutter pub add json_annotation equatable dio
flutter pub add dev:build_runner dev:json_serializable
```

TASK I

Generate and compare the Todo model

1. Define `Todo` with `userId`, `id`, `title`, `completed`, all final.
2. Annotate the class with `@JsonSerializable()` and add the `part` directive.
3. Add typed `fromJson` and `toJson` factory methods.
4. Run build_runner and confirm `todo.g.dart` appears.
5. Extend `Equatable` and list every field in `props`.
6. Build two identical todos and print `a == b`.

DONE WHEN

- [ ] `todo.g.dart` is generated and committed.
- [ ] `a == b` prints `true`.
- [ ] `flutter analyze` reports no issues.

REFERENCE

Code generation for Todo

```dart
part 'todo.g.dart';
@JsonSerializable()
class Todo {
  final int userId, id;
  final String title; final bool completed;
  const Todo({required this.userId, required this.id,
      required this.title, required this.completed});
  factory Todo.fromJson(Map<String, dynamic> j) => _$TodoFromJson(j);
  Map<String, dynamic> toJson() => _$TodoToJson(this);
}
```

The generator writes the boring half. Rebuilt from `todo.g.dart` after every model change.

REFERENCE

Value equality with Equatable

```
class Todo extends Equatable {
  final int userId, id;
  final String title;
  final bool completed;
  const Todo({required this.userId,
      required this.id, required this.title,
      required this.completed});
  @override
  List<Object?> get props => [userId, id, title, completed];
}
```

Same fields, same value. Without Equatable, `==` compares identity: two separate instances are never equal.

TASK II

Fetch todos, then retry with backoff

1. Fetch `/todos` from the API and map each item with `Todo.fromJson`.
2. Load the app's todo list from this call instead of typing entries by hand.
3. Wrap the call in `withRetry` with a fixed maximum number of attempts.
4. Grow the delay exponentially between attempts.
5. Add random jitter so retries do not line up.
6. Point at an invalid path and log each attempt with its delay.

DONE WHEN

- [ ] Todos render from the live API on launch.
- [ ] An invalid path retries several times with growing delays.
- [ ] The retry log is saved for the pull request.

REFERENCE

The API

```
GET https://jsonplaceholder.typicode.com/todos
GET https://jsonplaceholder.typicode.com/todos/1
GET https://jsonplaceholder.typicode.com/nope
# One element:
{
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
```

Field names match Todo exactly. No renaming needed. `/nope` returns 404: the failure case for Task II.

REFERENCE

Fetching the list

```
final _dio = Dio(BaseOptions(
  baseUrl: 'https://jsonplaceholder.typicode.com',
));
Future<List<Todo>> fetchTodos() async {
  final res = await _dio.get('/todos');
  final raw = res.data as List<dynamic>;
  return raw
      .map((j) => Todo.fromJson(j as Map<String, dynamic>))
      .toList();
}
```

Decode once, here. Nothing past this function ever sees raw JSON again.

REFERENCE

Backoff and jitter

```dart
Future<T> withRetry<T>(Future<T> Function() op) async {
  for (var i = 0; i < 5; i++) {
    try {
      return await op();
    } catch (_) {
      final b = 400 * (1 << i);
      await Future.delayed(Duration(milliseconds: b + Random().nextInt(b)));
    }
  }
  rethrow;
}
```

Retry what can succeed later. A 404 stays a 404.

TROUBLESHOOTING

Errors and fixes

`Target of URI hasn't been generated`	Run `build_runner` from Setup. Confirm `todo.g.dart` exists.
`Conflicting outputs were detected`	Add `--delete-conflicting-outputs` to the command.
`Undefined name _$TodoFromJson`	The `part` directive must name the file exactly: `todo.g.dart`.
`a == b is still false`	props is missing a field, or Todo does not extend Equatable.

Push before you leave.

Branch lab-4 · one commit per task · retry log saved under /screenshots